

Full-Stack Developer Task

Project Approval Workflow System

Objective

Develop a full-stack Laravel + Bootstrap/Vue.js/React(Choose any front-end framework) application that allows users to submit projects for approval, incorporating email notifications, stored procedures, and role-based access control.

Note: The same interface should serve for both project submission (by users) and project approval (by admins), with functionality adapting dynamically based on the authenticated user's role.

Requirements

1. Frontend (Bootstrap or Vue.js or React)

- **Login/Register Pages**
 - Authentication integrated with Laravel (Auth or Sanctum or Passport).
- **Project Submission Page**
 - Form to create new projects (fields: title, description, file uploads).
 - Real-time validation and UI feedback.
- **Dashboard**
 - Summary Statistics Panel
 - Total Submitted Projects: [count with percent]
 - Total Pending Projects: [count with percent]
 - Total Rejected Projects: [count with percent]
 - Total Approved Projects: [count with percent]
 - (Optional) Visual representation (charts/graphs)
 - Project List View
 - Table of all submitted projects showing:
 - Project name
 - Submitter details
 - Submission date
 - Current status (color-coded)
 - Last updated timestamp

- Admin-Specific Features
 - Action controls for each project:
 - Approve/Reject buttons
 - Reason input field (required for rejections)
 - Status change history
 - Bulk action capability
- Filter/sort functionality by:
 - Submission date
 - Project status
 - Submitter
- User Permissions
 - Regular users: View-only access to their submissions
 - Admin users: Full approval workflow controls

2. Backend (Laravel)

- **Schema Design:**

- Tables: `users`, `projects`, `approvals`, `audit_logs`.
- Relationships:
 - Each project belongs to a user.
 - Approvals are managed by admins.

- **Stored Procedure:**

- Create a MySQL stored procedure `sp_approve_project(project_id)` that:
 - Updates `projects.status` to "approved".
 - Logs the action in `audit_logs` with `timestamp`, `user_id`, and `action`.
 - Returns a success/failure status.

- **Test:** Invoke the procedure in Laravel using:

```
DB::select("CALL sp_approve_project(?)", [$id]);
```

A. Email Integration (Queues & Notifications)

- **Approval Emails:**

- Send a queued email to the submitter when:
 - A project is submitted.

- A project is approved or rejected (include a custom template with the reason).
- **Implement email templates** with dynamic variables for:
 - Project name
 - Approval status
 - Timestamp of action

B. API Endpoints (API Resources & Validation)

- **RESTful Routes:**
 - **POST /api/projects:** Submit a project (validate **title**, **description**, **files**).
 - **PATCH /api/projects/{id}/approve:** Approve a project (admin-only, uses stored procedure).
- **Response:**
- Return JSON responses using API Resources:
 - php artisan make:resource ProjectResource

C. Authentication & Authorization (Gates/Policies)

- **Roles:**
 - Users: Can submit projects.
 - Admins: Can approve projects.
- **Policies (Optional but Impressive):**
 - **ProjectPolicy:** Define an **approve()** method to restrict approval to admins.
- In **ProjectController@approve**, enforce authorization:
 - `$this->authorize('approve', $project);`

3. Bonus (Optional but Impressive)

- **Real-time Notification:**
 - Implement Laravel Echo with Pusher to display approval alerts.
- **Frontend Improvements:**
 - Use loading states, progress bars, and toasts to enhance UX.

✓ Evaluation Criteria

- **Full-Stack Proficiency**
Demonstrates a strong command of both frontend and backend development, including seamless integration between the two.
- **Code Structure**
Follows clean architecture with MVC and SOLID principles to ensure maintainability and scalability.
- **Performance Optimization**
Uses queues, efficient SQL queries, and optimized frontend components to deliver a responsive user experience.
- **Security Best Practices**
Implements proper input validation, role-based access control, and secure file handling mechanisms.

📦 Submission Guidelines

1. **Packaging Your Code**
Upload the files to a public GitHub repository and share the repository link.:
2. **Video Walkthroughs**
 - Create a **demo video** showcasing how your application works from a user's perspective.
 - Record a **code walkthrough** video where you explain your implementation, structure, and key logic.
 - Include both video links in your [README.md](#).